[IS216] Extra Exercises - Vue State Management & Routing

Objectives

- Gain a solid understanding of reactive programming concepts.
- Learn to leverage Vue framework features for building efficient web applications.
- Practice applying Vue directives to create dynamic and responsive web pages.

Instructions

- Questions without an asterisk are easy.
- Questions marked with * are moderately challenging.
- Questions marked with ** are challenging.
- Questions marked with *** are very challenging.

Contents

Question 1 – MiniMovie Favorites (*/**) 2

Question 2 – MiniCourse Planner (**/***) 8

Question 3 – MiniPrefs: Persistent App Preferences (**/***) 16

Note:

- If you notice any mistakes or errors in the questions, please email your instructors with a description of the issue. We will address it as soon as possible.
- For the best viewing experience, please use Microsoft Office 365 Online.

Question 1 – MiniMovie Favorites (*/**)

Background

You are tasked with building a small SPA-style app where users can browse movies and maintain a Favorites list using Pinia state management. The same state is shown/used in multiple components (Navbar badge + movie cards + favorites page), so you need to use Pinia stores (state/getters/actions) and component communication.

Features to implement

Feature 1 — Favorites store (Pinia)

Create a Pinia store useFavoritesStore that manages favorites.

- State
  - favorites: array of movie objects: { id, title, genre, rating }
- Getters
  - count: number of favorite movies
  - averageRating: average rating of favorites (0 if none)
- Actions (must be called from events like @click)
  - add(movie)
  - remove(movieId)
  - clearAll()

Feature 2 — Browse movies (MoviesView)

Display a list of movies (use a hardcoded array in the view or store).

- Each movie shows:
  - title
  - genre
  - rating (1–10)
- Each movie has a button:
  - “Add to Favorites” if not in favorites
  - “Remove” if already in favorites

Feature 3 — FavoritesView (shared state + derived info)

Create a FavoritesView.vue that displays:

- a list of favorite movies
- the averageRating getter
- a “Clear All” button that calls clearAll()

Feature 4 — Filtering movies

In MoviesView, add a <select> for genre filter:

- options: All, Action, Comedy, Drama, Sci-Fi (or any genres you define)
- When the selection changes, filter the displayed list.
- Hint: use v-model + computed filtered array

Project Structure

```
src/
|-- main.js
|-- App.vue
|-- router/index.js
|-- stores/
```

```
      |-- favorites.js (to modify)
|-- components/
      |-- NavBar.vue
      |-- MovieCard.vue
|-- views/
      |-- MoviesView.vue (to modify)
      |-- FavoritesView.vue (to modify)
```

Setup Instructions:

1. Navigate to the `q1` directory in your terminal.
2. Run the command "`pnpm install`" to install the required dependencies.
3. Run the command "`pnpm dev`" to run the app in your development server.

Part A: Implement Feature 1 — Favorites store (Pinia)

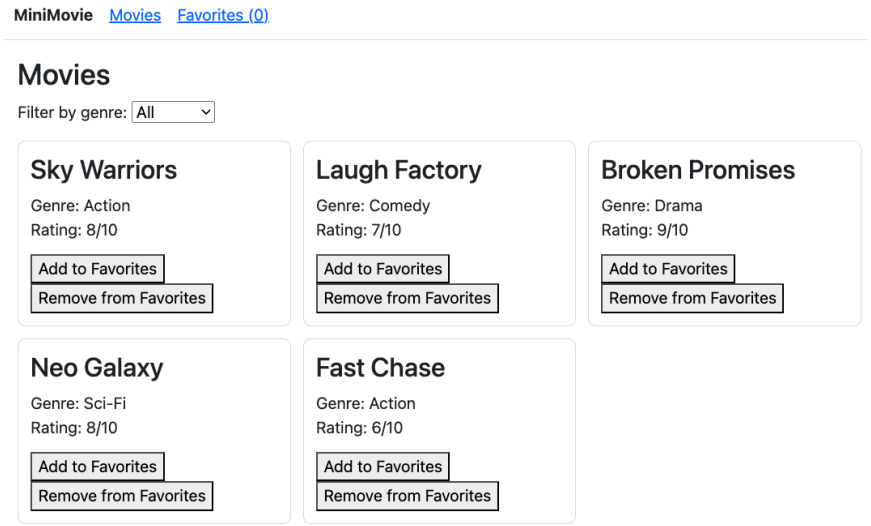
1. Navigate to the `q1/src/stores` directory.
2. Open the `favorites.js` file and add or modify the code (indicated in `TODO`) to fulfill the requirements described above.

Part B: Implement Feature 2 — Browse movies (MoviesView)

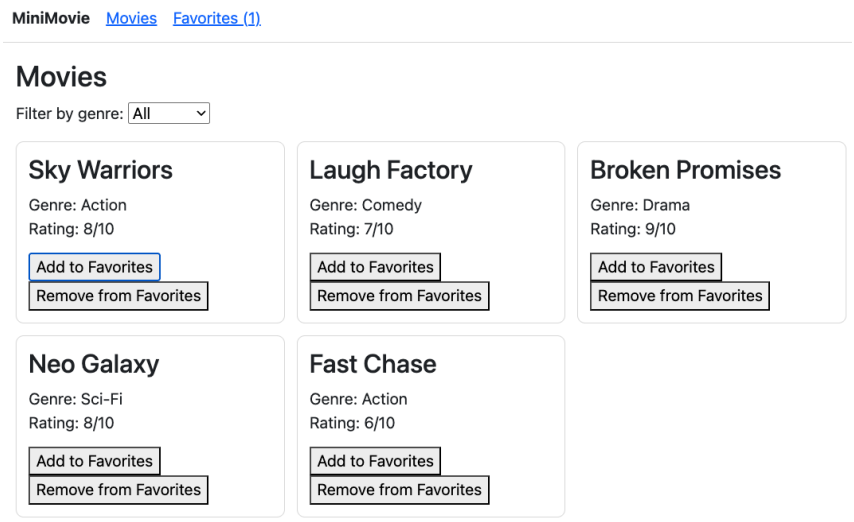
1. Navigate to the `q1/src/views` directory.
2. Open the `MoviesView.vue` file and add or modify the code (indicated in `TODO`) to fulfill the requirements described above. Use MovieCard component from `/src/components/MovieCard.vue`

If Part A and Part B are implemented correctly, the app should behave as follows:

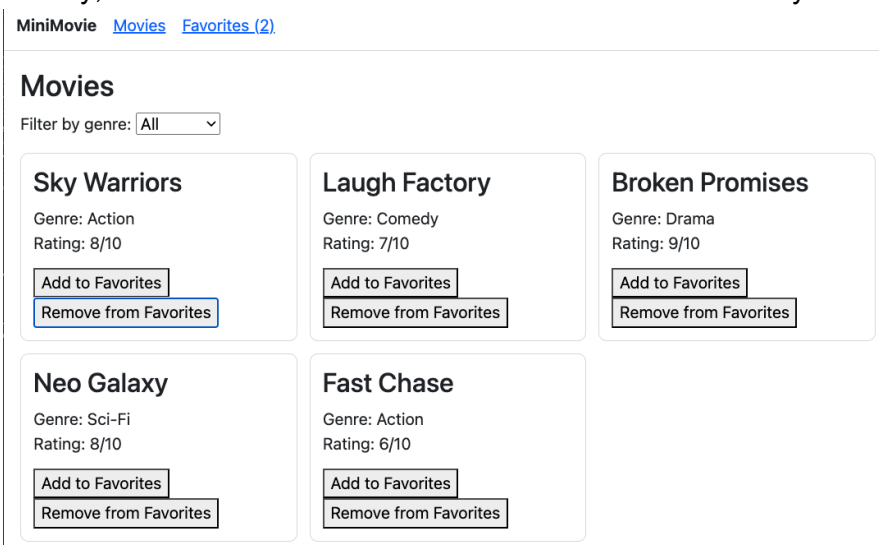
1. When the app is loaded in the browser initially



2. When the user clicks "Add to Favorites" of the movie "Sky Warriors"



3. After the user clicks “Add to Favorites” of other movies “Broken Promises” and “Neo Galaxy, and clicks “Remove from Favorites” of the movie “Sky Warriors”

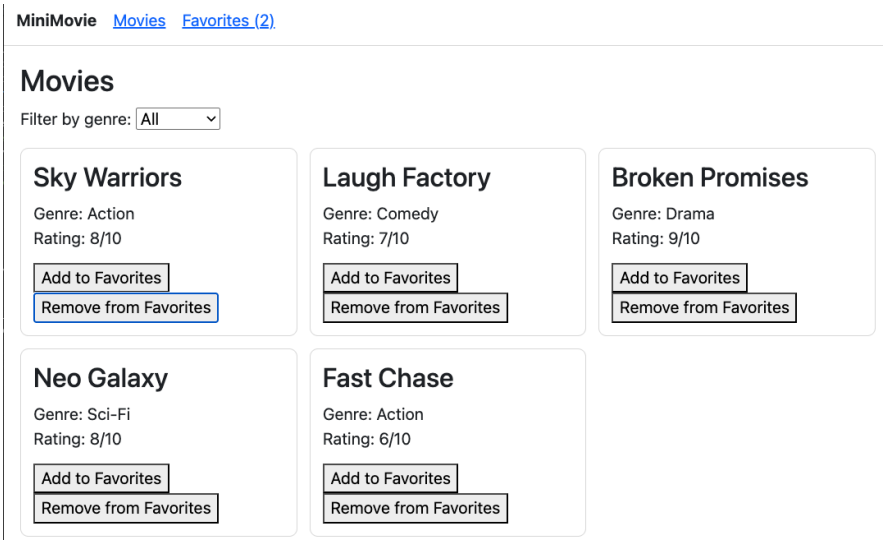


Part C: Implement Feature 3 — FavoritesView (shared state + derived info)

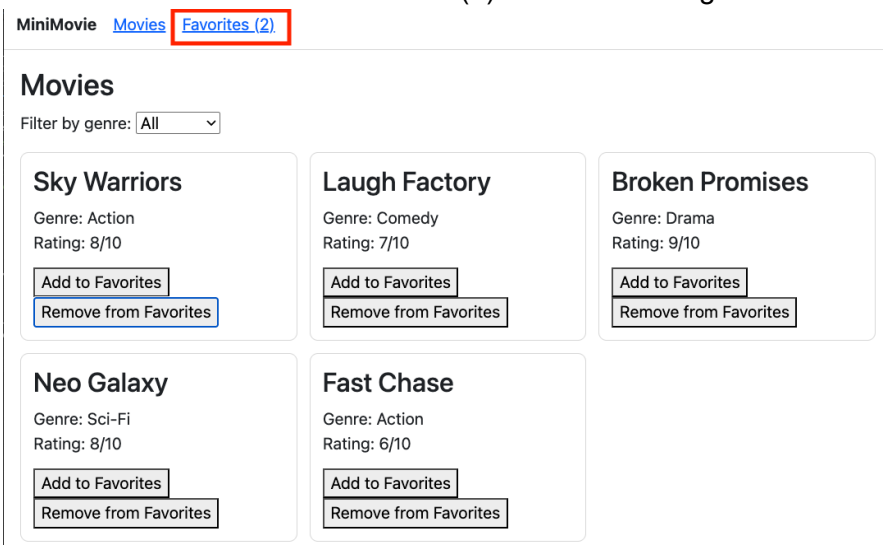
1. Navigate to the `q1/src/views` directory.
2. Open the `FavoritesView.vue` file and add or modify the code (indicated in `TODO`) to fulfill the requirements described above.

If Part C is implemented correctly, the app should behave as follows:

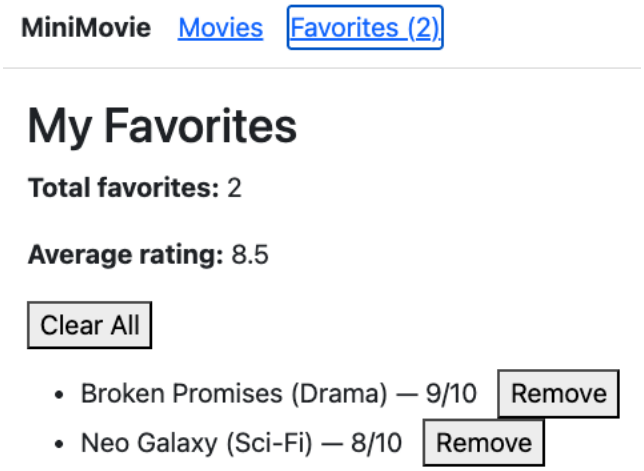
1. After the app is loaded in the browser, and after the user clicks “Add to Favorites” of the movies “Broken Promises” and “Neo Galaxy:



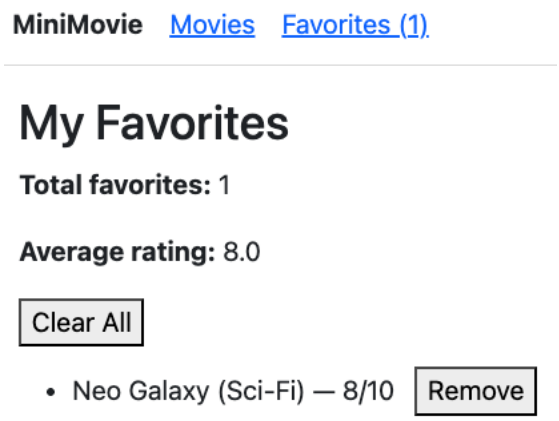
2. The user then clicks the “Favorites (2)” link in the navigation bar



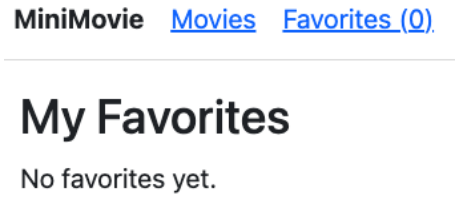
3. The app shows the favorite movies:



4. The user is able to “Clear All” or “Remove” individual favorite movies. For example, the user clicks “Remove” button for “Broken Promises”



5. The user then clicks “Clear All” button:

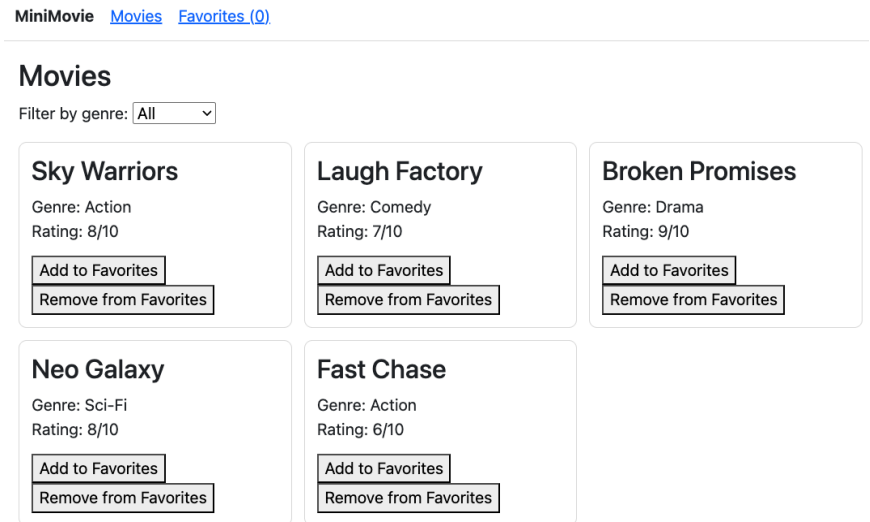


Part D: Implement Feature 4 — Filtering movies (MoviesView)

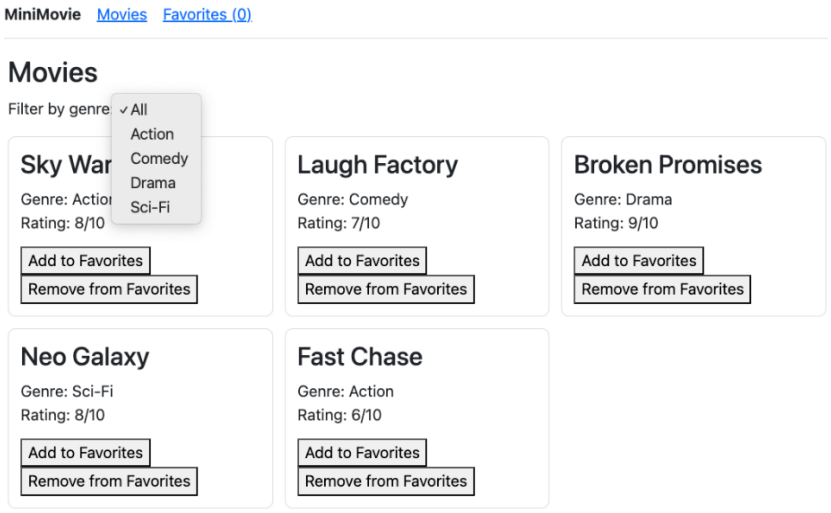
1. Navigate to the `q1/src/views` directory.
2. Open the `MoviesView.vue` file and add or modify the code you completed in Part B to display only the movies filtered by genre.

If Part D is implemented correctly, the app should behave as follows:

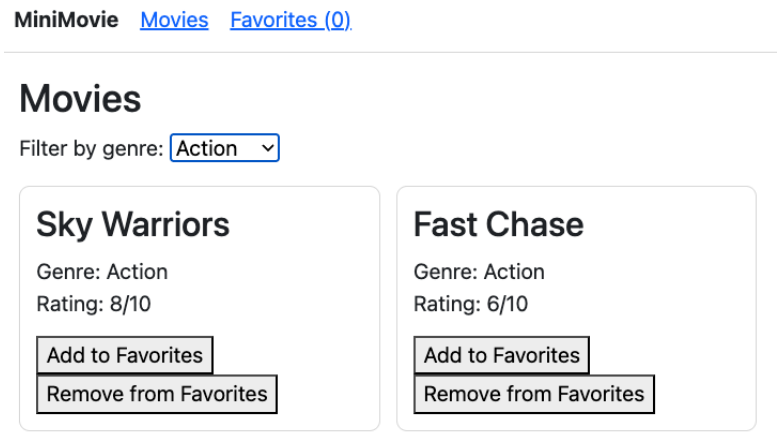
1. After the app is loaded in the browser initially



2. The user can filter by genre:



3. For example, the user selects “Action” genre:



Question 2 – MiniCourse Planner (**/***)

Background

You are tasked with building a university course planner where students add courses to a plan and must "log in" to access protected pages. To implement this, you need to use a state management solution (Pinia) and routing with navigation guards.

Features to implement

Feature 1 — Login + redirect after login

- After login, user is redirected back to the originally requested route using route.query.redirect

Feature 2 — State Management (Pinia)

- auth store: Create a Pinia store that stores login data (already implemented)
  - If the username is "admin", then the user role is "admin", otherwise the role is "student".
  - Token value is a fake token value used to simulate the user's logged in state. That is, if the token value is set, then the user is logged in. If it is null, then the user is not logged in.
- plan store: Create a Pinia store that stores the planned courses (totalCredits, add/remove/toggle)
  - totalCredits -> compute the total credits of the planned courses.
  - add(course) -> check if the course already exists. If it doesn't exist, add this course information (object) into the existing list of planned courses (planned state).
  - remove(course) -> remove the course from the planned courses.
  - toggle(course) -> if the course already exists, remove it from the planned courses. Otherwise, add it to the planned courses.

Feature 3 — Navigation Guards

- If route needs login and user is not logged in → redirect to /login?redirect=<original>
- If route needs admin role and user is not admin → and alerts "Access denied" and then redirect to /courses

Feature 4 — Shared State across Components

- Navbar shows links to /courses, /plan, and /admin
- Navbar displays Planned (X) using plan.count
- Navbar shows login status and logout button, i.e.,
  - If the user is logged in, it shows "Hello, student (student)" and "Logout" button
  - Otherwise, it just shows the "Login" link

Feature 4 — Course Details

- Display the details of the course

Project Structure

```
src/
|-- main.js
|-- App.vue
|-- router/index.js (to modify)
|-- stores/
    |-- auth.js
    |-- plan.js (to modify)
|-- components/
    |-- NavBar.vue (to modify)
    |-- CourseCard.vue
|-- views/
    |-- AdminView.vue
    |-- CourseDetailView.vue (to modify)
    |-- CoursesView.vue
    |-- LoginView.vue (to modify)
    |-- PlanView.vue
```

Routing:

- / redirects to /courses
- /courses shows a list of courses
- /courses/:code shows a course detail page (route param)
- /plan shows the student's planned courses (protected: requiresAuth)
- /admin shows admin page (protected: requiresAuth + requiresRole="admin")
- /login shows login page

Setup Instructions:

1. Navigate to the `q2` directory in your terminal.
2. Run the command "`pnpm install`" to install the required dependencies.
3. Run the command "`pnpm dev`" to run the app in your development server.

Part A: Implement Feature 1 — Login + redirect after login

1. Navigate to the `q2/src/views` directory.
2. Open the `LoginView.vue` file and add or modify the code (indicated in `TODO`) to fulfill the requirements described above.

Part B: Implement Feature 2 — State Management (Pinia)

1. Navigate to the `q2/src/stores` directory.
2. Open the `plan.js` file and add or modify the code (indicated in `TODO`) to fulfill the requirements described above.

Part C: Implement Feature 3 — Navigation Guards

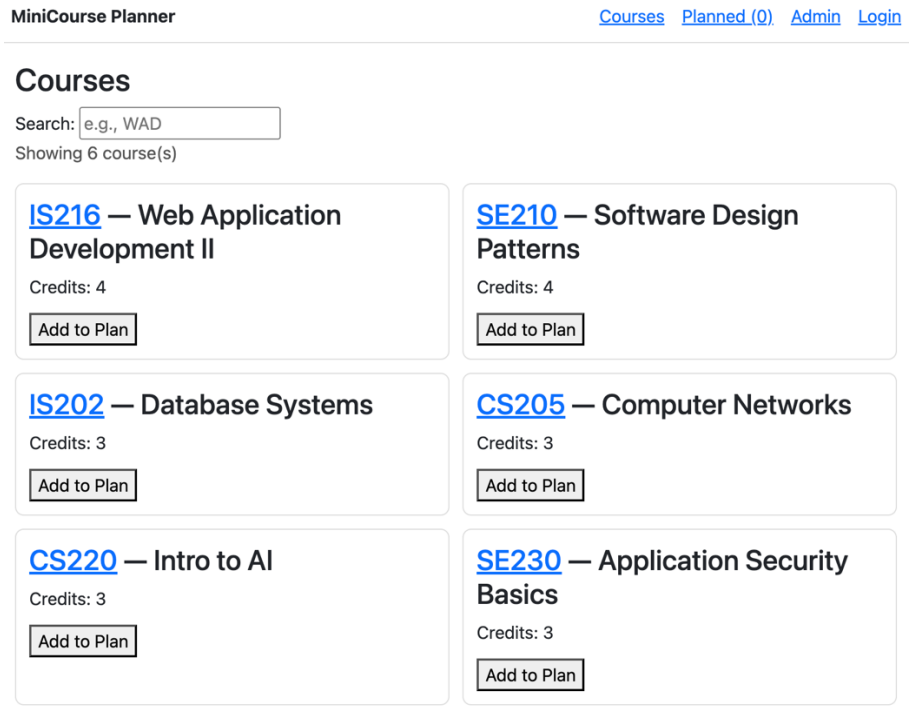
1. Navigate to the `q2/src/router` directory.
2. Open the `index.js` file and add or modify the code (indicated in `TODO`) to fulfill the requirements described above.

Part D: Implement Feature 4 — Shared State across Components

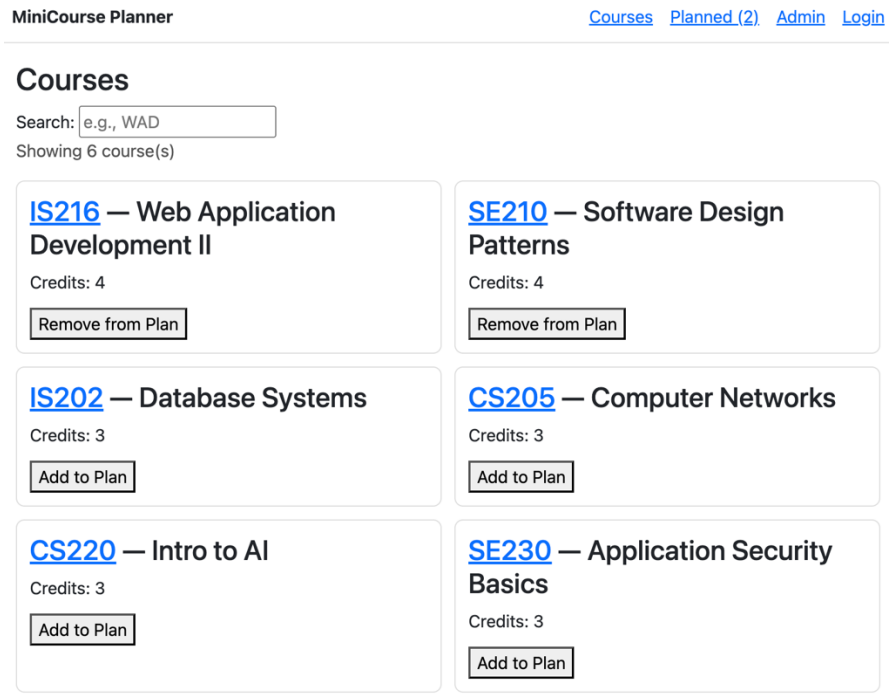
1. Navigate to the `q2/src/components` directory.
2. Open the `NavBar.vue` file and add or modify the code (indicated in `TODO`) to fulfill the requirements described above.

If Part A, Part B, Part C, and Part D are implemented correctly, the app should behave as follows:

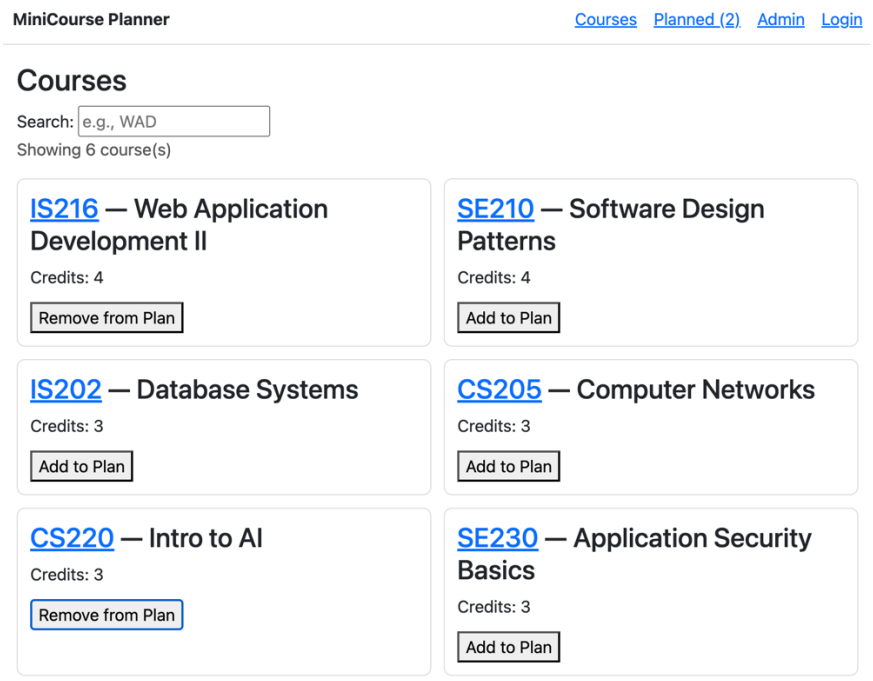
1. When the app is loaded in the browser initially



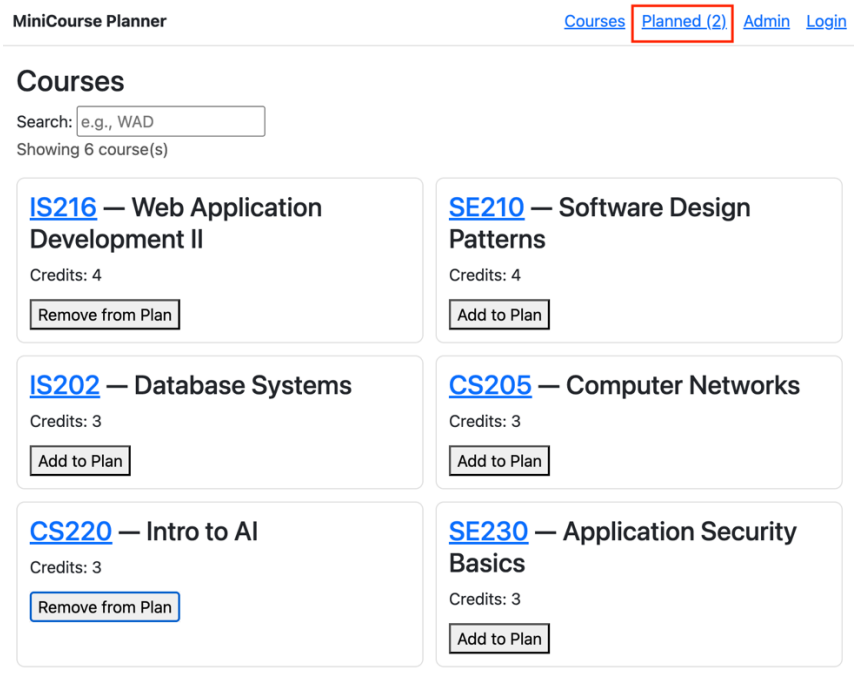
2. When the user clicks “Add to Plan” of the courses IS216 and SE210



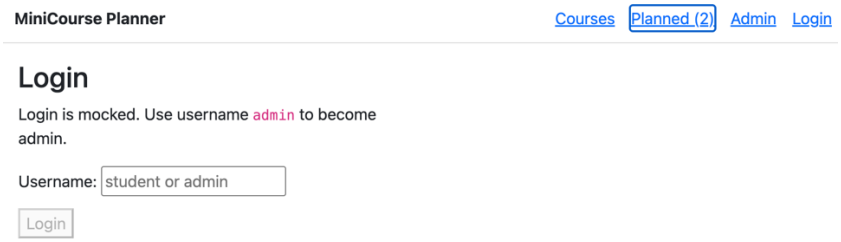
3. When the user clicks “Remove from Plan” of the course SE210 and clicks “Add to Plan” of the course CS220



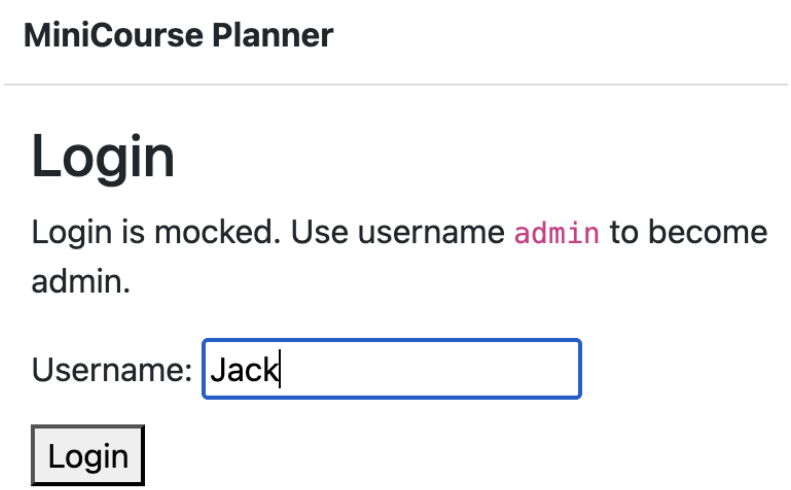
4. The user clicks the link “Planned (2)” in the navigation bar



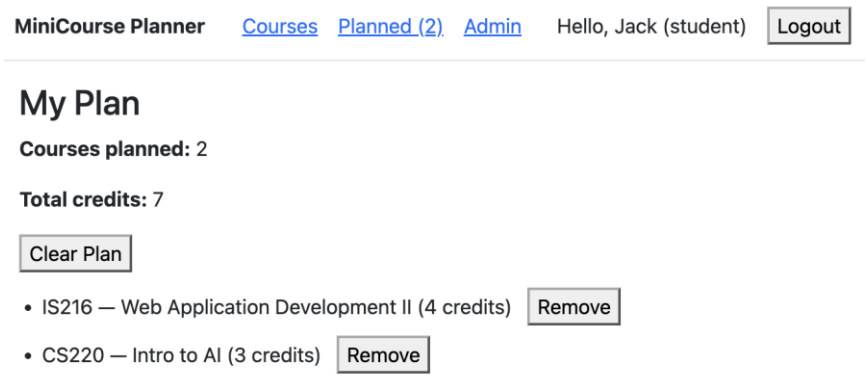
5. The app redirects to the login page as the user has not logged in:



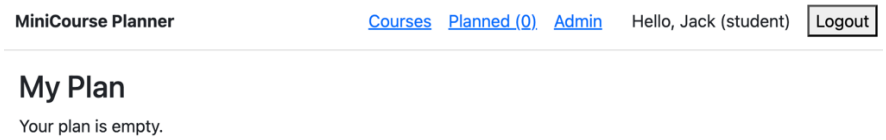
6. The user enters “Jack” for the username and clicks “Login”



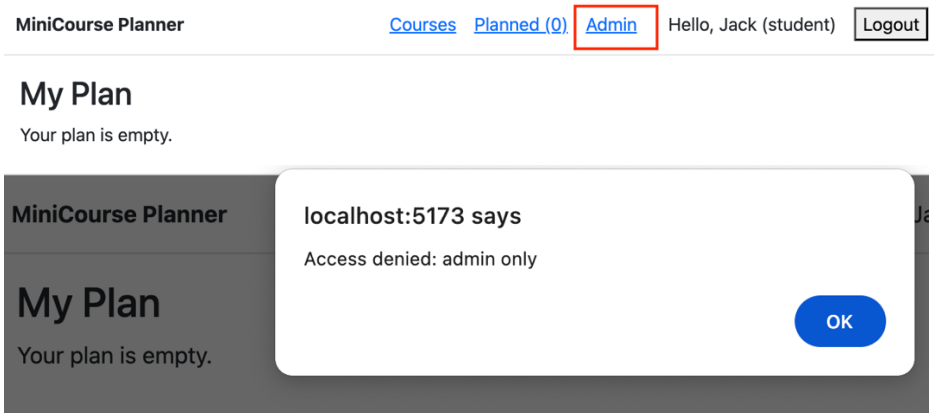
7. Once the user has logged in (above), the app redirects to /plan page:



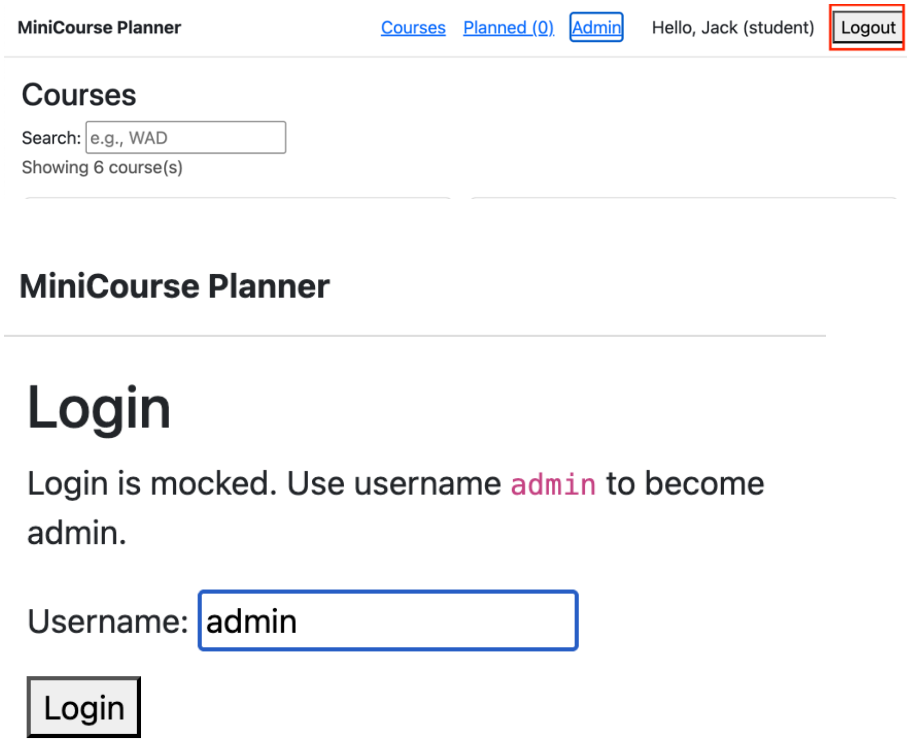
8. The user clicks “Clear Plan” button:



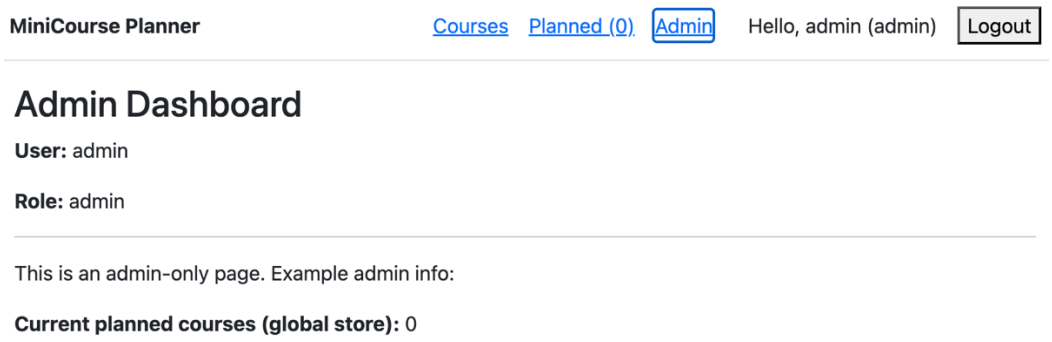
9. When the user then clicks the link “Admin” in the navigation bar, it shows an alert (as the user is not an admin):



10. The user then clicks the “Logout” button in the navigation bar and then log in as an “admin”:



11. As an admin, the user can go to the admin page, by clicking the link “Admin” in the navigation bar

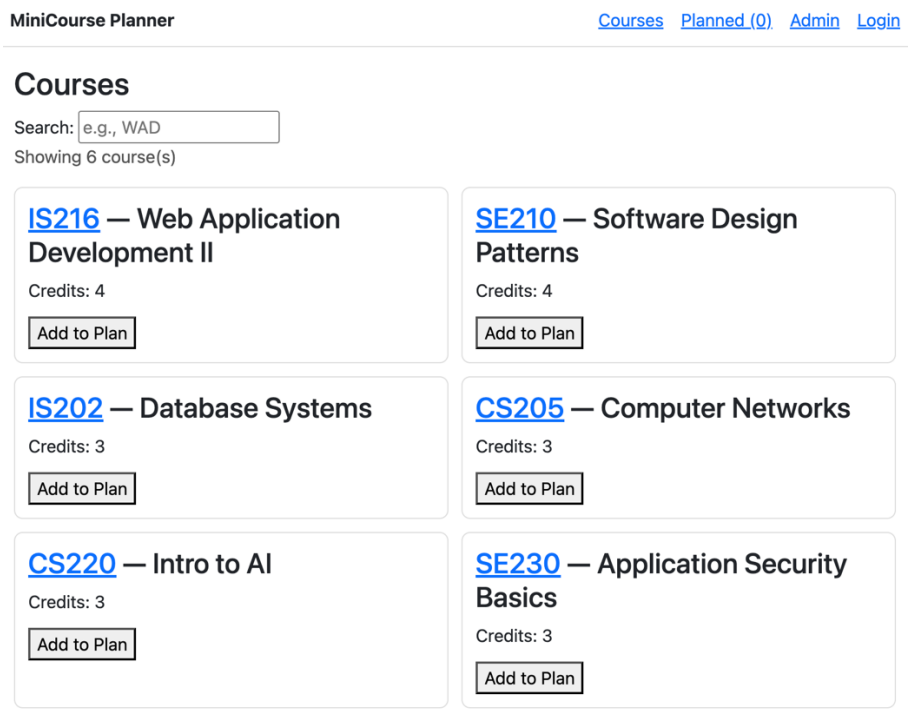


Part E: Implement Feature 5 — Course detail

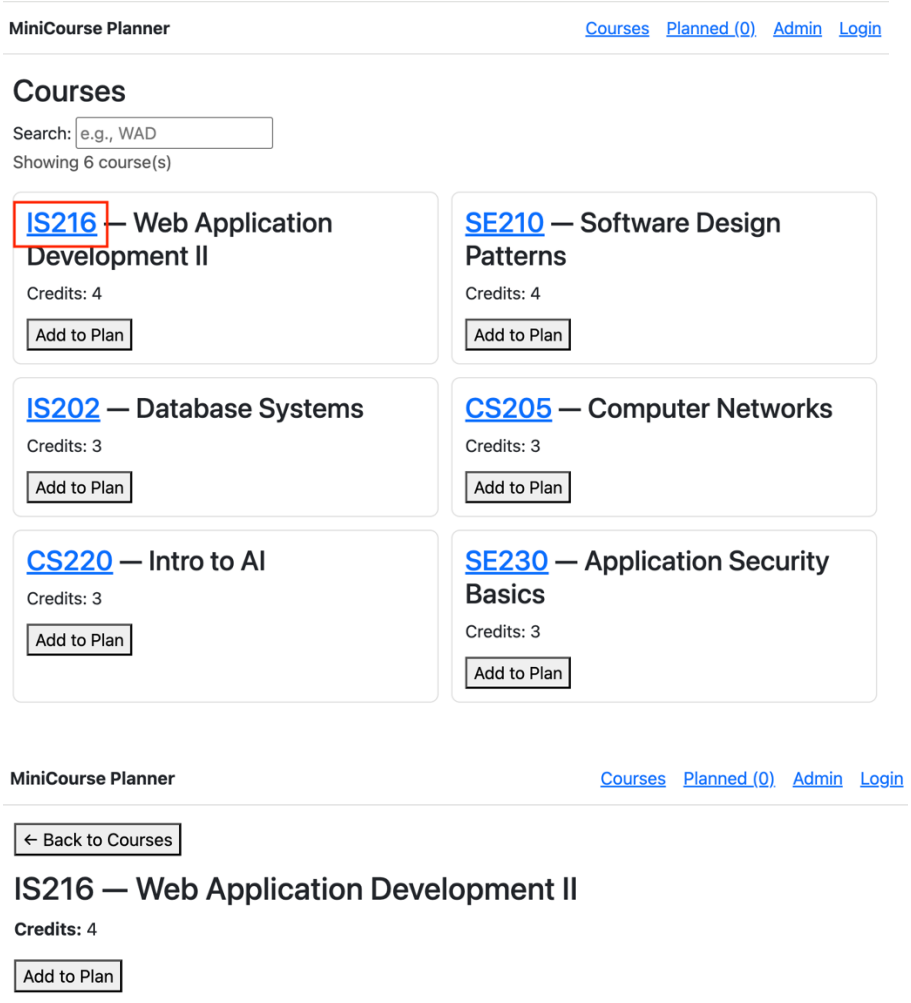
1. Navigate to the `q2/src/views` directory.
2. Open the `CourseDetailView.vue` file and add or modify the code (indicated in `TODO`) to fulfill the requirements described above.

If Part E is implemented correctly, the app should behave as follows:

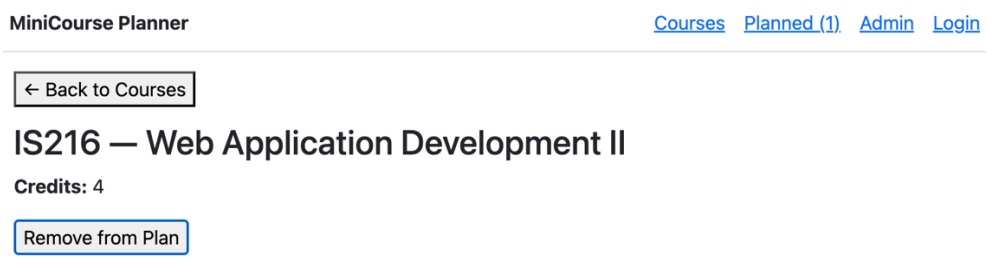
1. When the app is loaded in the browser initially



2. When the user clicks on the link “IS216”, the detail of IS216 is shown:



3. In this course detail page, the user can click “Add to Plan” button to add the course to the plan. In this case, “Remove from Plan” button appears, which allows the user to remove the course from the plan.


MiniCourse Planner
Courses
Planned (1)
Admin
Login
← Back to Courses
IS216 — Web Application Development II
Credits: 4
Remove from Plan

Question 3 – MiniPrefs: Persistent App Preferences (**/***)

Background

You are tasked with building a small SPA that remembers user preferences across browser sessions. The app has a light/dark theme, a collapsible sidebar, and a language switcher (English / 中文 / Bahasa). All three preferences must be stored in a single Pinia store and must survive a full-page reload. The same store is consumed by the Navbar, the Sidebar, the Settings page, and the root App component so that changes are immediately visible throughout the app.

Features to implement

Feature 1 — Preferences store (via Pinia)

Create a Pinia store `usePreferencesStore`.

- State
  - `theme`: 'light' | 'dark' (default: 'light')
  - `sidebarCollapsed`: boolean (default: false)
  - `language`: 'en' | 'zh' | 'ms' (default: 'en')
- Getters
  - `isDark`: true when theme === 'dark'
  - `currentLanguageLabel`: human-readable label of the current language
- Actions
  - `setTheme(theme)`
  - `toggleTheme()`
  - `setSidebarCollapsed(collapsed)`
  - `toggleSidebar()`
  - `setLanguage(lang)`

Feature 2 — Persistence (via Vue localStorage)

- On every change to any preference, save the whole preferences object to Vue `localStorage` under the key "miniprefs".
- When the store is created, read the saved object from Vue `localStorage` (if present) and initialise the state with those values.
- A full-page refresh must restore the last chosen theme, sidebar state and language.

Feature 3 — Global Theme Application

- In `App.vue`, watch the theme (or `isDark`) and add/remove a CSS class "dark" on the `<html>` (or `<body>`) element.
- Provide CSS variables (or simple class-based styles) so that the whole app switches between light and dark appearance.

Feature 4 — Sidebar that Reacts to Store State

- The Sidebar component shows a narrow icon-only mode when `sidebarCollapsed` is true, otherwise a full-width mode with labels.
- A toggle button (in the Navbar or inside the Sidebar) calls `toggleSidebar()`.

Feature 5 — Language Switcher & Translated Strings

- A language selector (select or buttons) calls `setLanguage()`.

- At least the following UI strings must change according to the current language:
  - Navbar brand / title
  - Sidebar menu item labels
  - Settings page headings and descriptions
- You may hard-code a small translation map inside the store or a composable; full i18n libraries are not required.

Feature 6 — Settings Page

- A dedicated Settings view that lets the user change all three preferences (theme, sidebar, language) and shows the current values.
- A "Reset to defaults" button that restores the three defaults and clears the Vue `localStorage` entry.

Project Structure

```
src/
|-- main.js
|-- App.vue (to modify – theme class)
|-- router/index.js
|-- stores/
    |-- preferences.js (to modify)
|-- components/
      |-- NavBar.vue (to modify)
      |-- Sidebar.vue (to modify)
|-- views/
      |-- HomeView.vue
      |-- SettingsView.vue (to modify)
```

Setup Instructions:

1. Navigate to the q3 directory in your terminal.
2. Run the command "`pnpm install`" to install the required dependencies.
3. Run the command "`pnpm dev`" to run the app in your development server.

Part A: Implement Feature 1 — Preferences Store (Pinia)

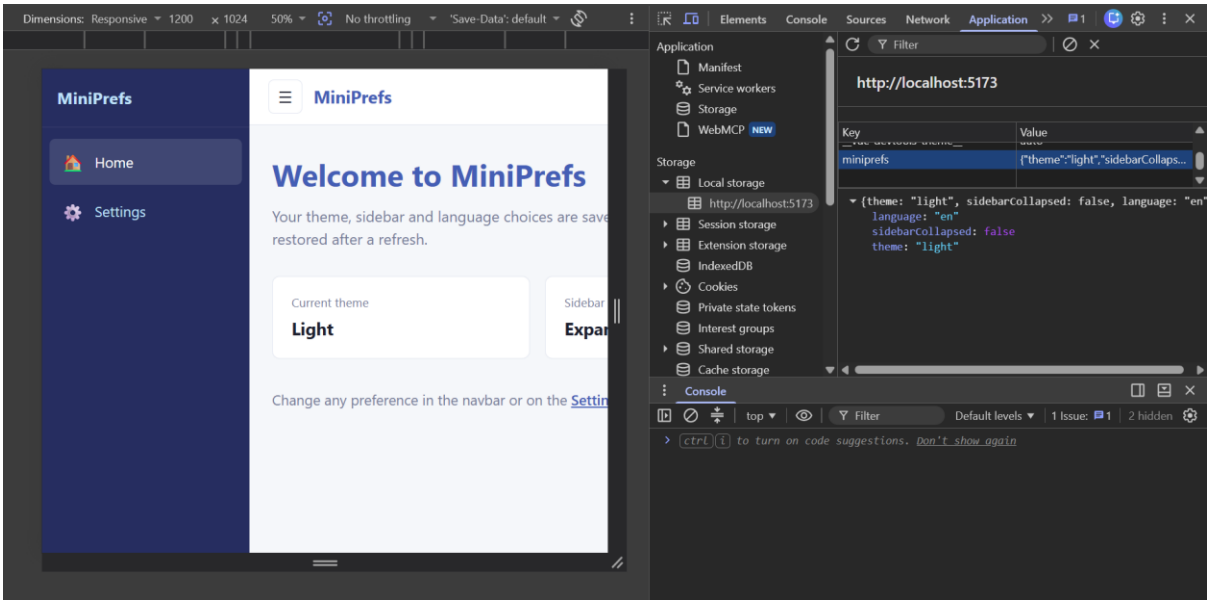
1. Navigate to the `q1/src/stores` directory.
2. Open `preferences.js` and implement the store using Pinia `defineStore`. Complete all actions listed in Feature 1. Both state and getters have been provided.

Part B: Implement Feature 2 — Persistence (Vue `localStorage`)

1. Still in `preferences.js`, add logic that:
   - Loads saved preferences from Vue `localStorage` when the store is created.
   - Saves the current preferences object to Vue `localStorage` whenever any value changes (you may use a watcher inside the store or save inside each action).
2. Verify that a hard refresh of the browser restores the last values.

If Part A and Part B are implemented correctly:

1. Changing any preference and then refreshing the page restores the same values.
2. Opening the browser's Developer Tools → Application → Local Storage tab shows a key named "miniprefs".

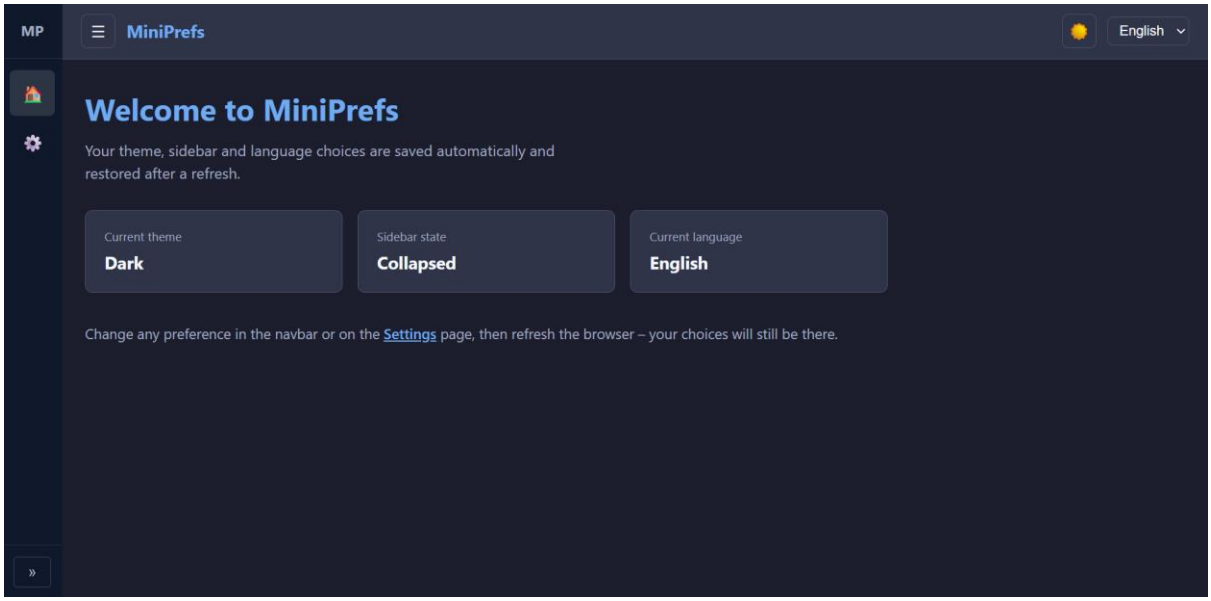


Part C: Implement Feature 3 & 4 — Theme + Sidebar

1. In `App.vue`, apply a "dark" class on the document element whenever the theme is dark.
2. Complete `Sidebar.vue` so that it collapses / expands according to `sidebarCollapsed` and provides a toggle control.
3. Complete `NavBar.vue` with a theme toggle button that calls `toggleTheme()`.

If Part C is implemented correctly:

1. Clicking the theme toggle (🌙 on the right of the top navbar) instantly switches the whole UI between light and dark styles.
2. Clicking the sidebar toggle (« at the bottom of the side bar) narrows the sidebar to icons only (or » expands it again).
3. Both settings survive a page refresh and update the `preferences` pinia store.

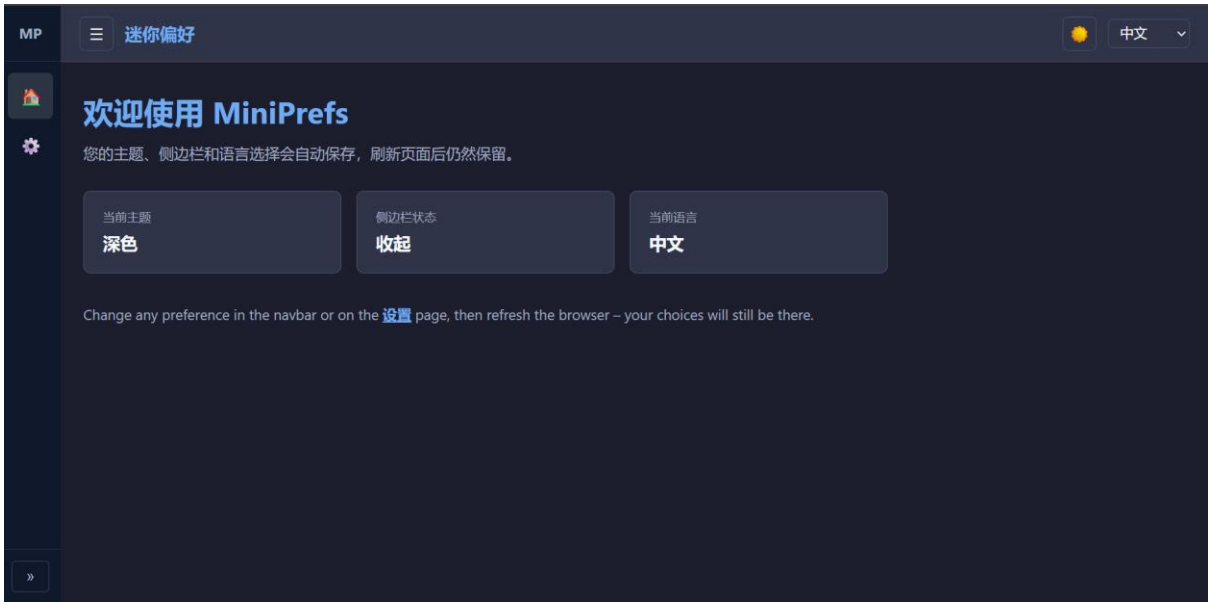

MP
MiniPrefs
English
Welcome to MiniPrefs
Your theme, sidebar and language choices are saved automatically and restored after a refresh.
Current theme
Dark
Sidebar state
Collapsed
Current language
English
Change any preference in the navbar or on the Settings page, then refresh the browser – your choices will still be there.

Part D: Implement Feature 5 & 6 — Language + Settings Page

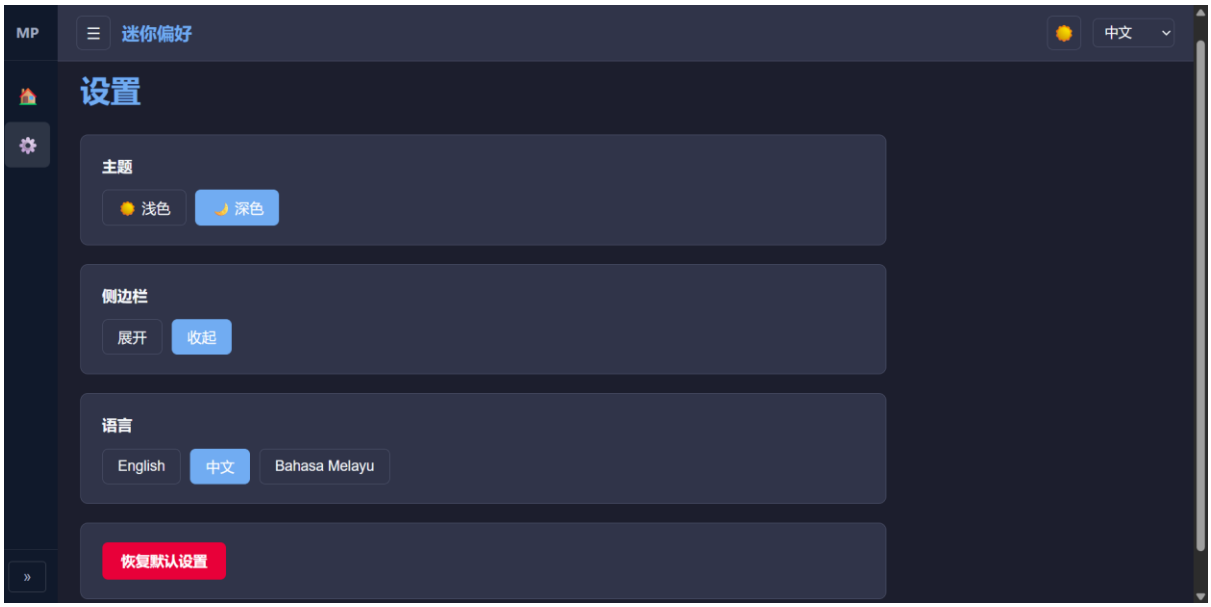
1. Add a language selector in the `Navbar.vue` and the `SettingsView.vue` that calls `setLanguage()`.
2. Make the listed UI strings reactive to the current language.
3. Complete `SettingsView.vue` so that a user can change theme, sidebar and language, and can reset everything to defaults and update the `preferences` pinia store.

If Part D is implemented correctly:

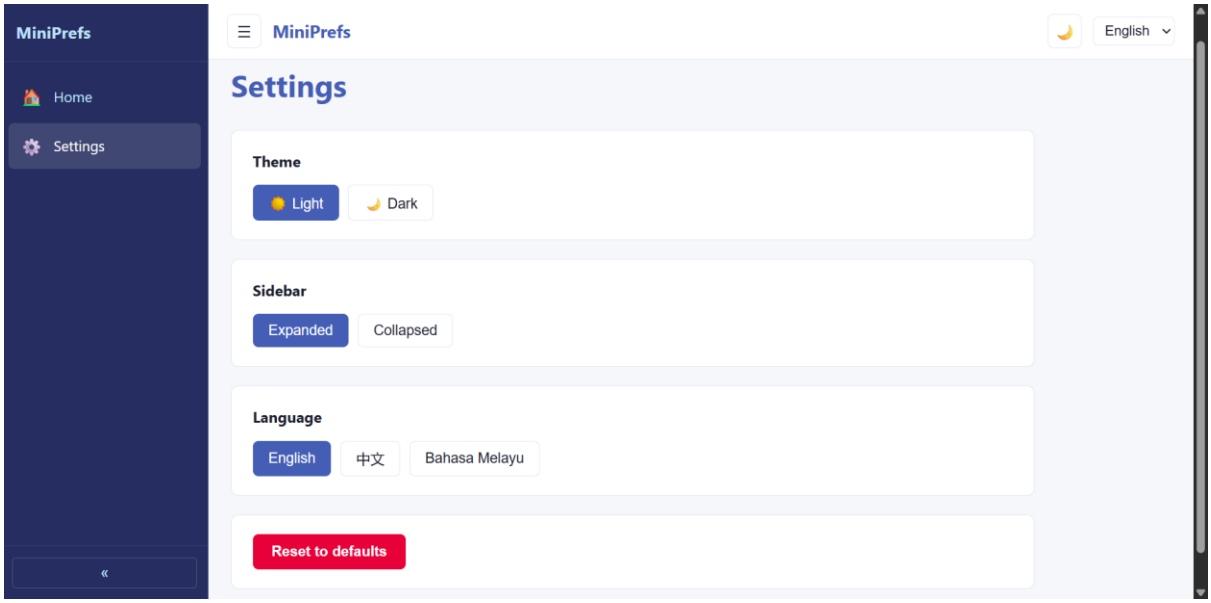
1. Switching language updates the Navbar title, sidebar labels and Settings page text immediately.



2. The Settings page shows the current values.



3. Clicking on the Reset button restores defaults and clears Vue `localStorage`.



4. All three preferences remain consistent across Home and Settings views and after a refresh.

Part E (Advanced): Prefer a Composable or Plugin-Style Persistence

*This part is **optional** but recommended for cleaner architecture.*

1. Extract the Vue `localStorage` load/save logic into a small composable (e.g. `usePersistentRef`) or use the official `pinia-plugin-persistedstate` package.
2. Demonstrate that the store code itself becomes shorter and that adding a new preference requires almost no extra persistence code.

Tips & Common Pitfalls

- Use the Pinia syntax for Vue Composition API: `defineStore('preferences', () => { … })`.
- When reading from Vue `localStorage`, always wrap `JSON.parse` in a `try/catch` – corrupted data should not crash the app.
- Apply the dark class on `document.documentElement` (the `<html>` tag) so that CSS variables cascade correctly.
- Keep the translation map simple – a plain object keyed by language code is sufficient.
- Test the full cycle: change → refresh → verify → change again → refresh again.

~ End of Exercises ~